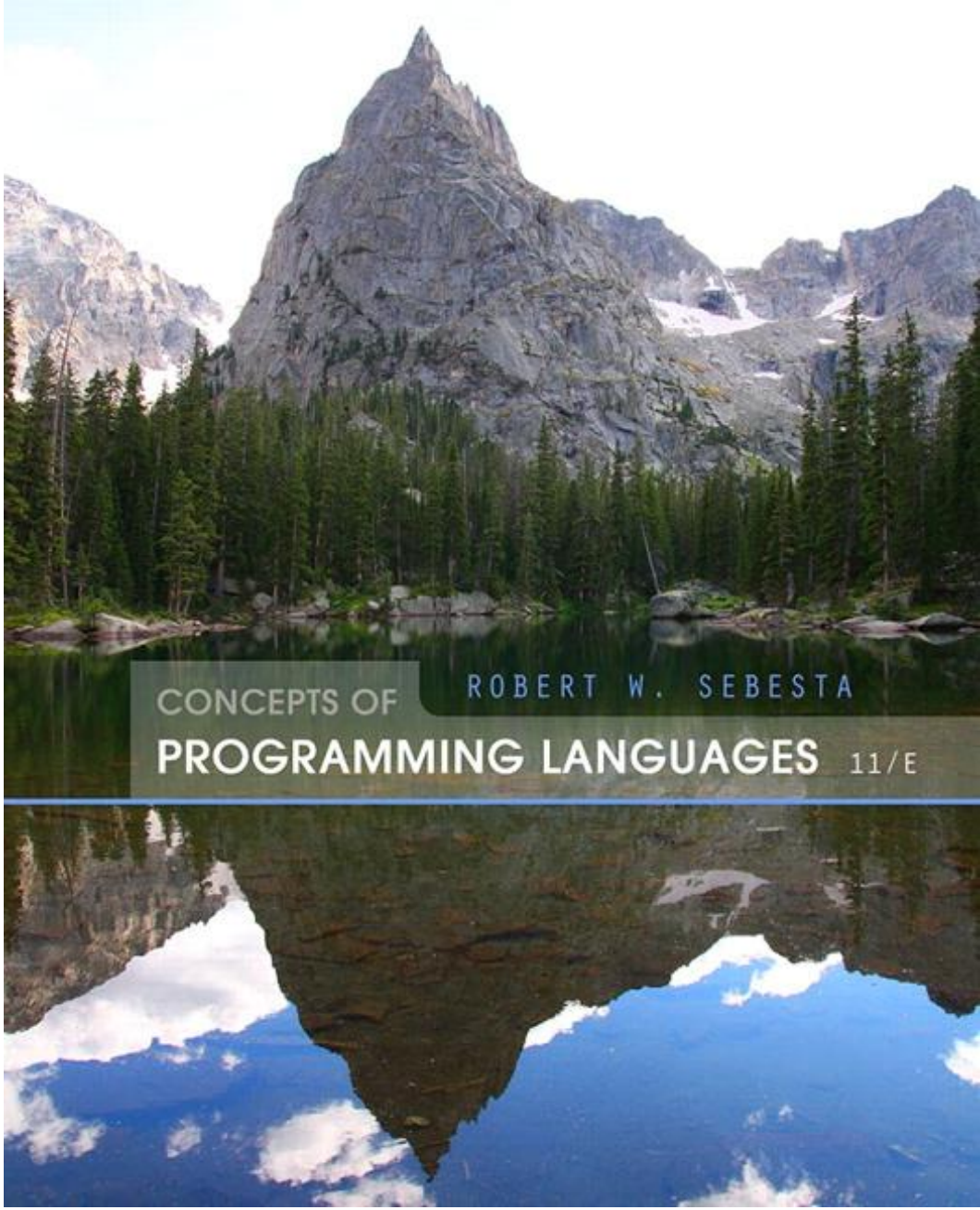
Bölüm 5

İsimler, Bağlayıcılar ve Kapsamlar (Names, Bindings, and Scopes)



ISBN 0-321-49362-1

Bölüm 5 Konular

- 5.1 Giriş (Introduction)
- 5.2 İsimler (Names)
- 5.3 Değişkenler (Variables)
- 5.4 Bağlayıcılık Kavramı (The Concept of Binding)
- 5.5 Kapsam (Scope)
- 5.6 Kapsam ve Ömür (Scope and Lifetime)
- 5.7 Referans Ortamları (Referencing Environments)
- 5.8 İsimlendirilmiş Sabitler (Named Constants)

Copyright © 2015 Pearson. All rights reserved.

5.1 Giriş (Introduction)

- İmperatif diller, von Neumann mimarisinin soyutlamalarıdır.
  - Bellek
  - İşlemci
- Değişkenler özelliklerle (attributes) karakterize edilir.
  - Bir tür tasarlamak için kapsam, ömür, tür denetimi, başlatma ve tür uyumluluğu dikkate alınmalıdır. (To design a type, must consider scope, lifetime, type checking, initialization, and type compatibility)

Copyright © 2015 Pearson. All rights reserved.

5.2 Names

- Names için tasarım sorunları:
  - Names büyük/küçük harfe duyarlı (case sensitive) mı?
  - Özel kelimeler ayrılmış kelimeler mi yoksa anahtar kelimeler mi? (Are special words reserved words or keywords?)

Copyright © 2015 Pearson. All rights reserved.

5.2 Names (Devam Ediyor)

- Uzunluk
  - Çok kısa olursa, çağrışım yapamazlar.
  - Dil örnekleri:
    - C99: sınırlama yoktur, ancak yalnızca ilk 63 karakter önemlidir; ayrıca, harici isimler (external names) en fazla 31 karakterle sınırlıdır.
    - C# ve Java: sınır yok ve hepsi önemlidir
    - C++: sınırlama yoktur, ancak uygulayıcılar genellikle bir sınırlama getirir.

Copyright © 2015 Pearson. All rights reserved.

5.2 Names (Devam Ediyor)

- Özel karakterler
  - PHP: tüm değişken isimleri dolar işaretiyle başlamalıdır
  - Perl: tüm değişken isimleri, değişkenin türünü belirten özel karakterlerle başlar
  - Ruby: @ ile başlayan değişken isimleri instance değişkenleridir; @@ ile başlayanlar ise class değişkenleridir

Copyright © 2015 Pearson. All rights reserved.

5.2 Names (Devam Ediyor)

- Büyük/küçük harf duyarlılığı (Case sensitivity)
  - Dezavantaj: okunabilirlik (benzer görünen isimler farklıdır)
    - C tabanlı dillerde isimler büyük/küçük harfe duyarlıdır
    - Diğerleri böyle değildir
    - C++, Java ve C#'da daha kötüdür çünkü önceden tanımlanmış isimler büyük ve küçük harf karışık olarak yazılır (örneğin IndexOutOfBoundsException).

Copyright © 2015 Pearson. All rights reserved.

5.2 Names (Devam Ediyor)

- Özel kelimeler
  - Okunabilirliği kolaylaştırmak için; cümle öbeklerini sınırlandırmak veya ayırmak için kullanılır.
  - *Anahtar kelime (keyword)*, yalnızca belirli bağlamlarda özel olan bir kelimedir.
  - *Rezerve edilmiş kelime (reserved word)*, kullanıcı tanımlı isim olarak kullanılamayan özel bir kelimedir.
  - Rezerve edilmiş kelimelerle ilgili olası sorun: Çok fazla sayıda varsa, birçok çakışma meydana gelir (örneğin, COBOL'da 300 rezerve edilmiş kelime vardır!).

Copyright © 2015 Pearson. All rights reserved.

5.3 Variables

- Variable, bir bellek hücresinin soyutlamasıdır.
- Değişkenler (variables), altı öznitelik grubu olarak tanımlanabilir:
  - Name
  - Address
  - Value
  - Type
  - Lifetime
  - Scope

Copyright © 2015 Pearson. All rights reserved.

5.3.1 Değişken Öznitelikleri (Variables Attributes)

- Name - tüm değişkenlerde bunlar bulunmaz
- Address - ilişkili olduğu bellek adresi
  - Bir değişken, yürütme sırasında farklı zamanlarda farklı adreslere sahip olabilir.
  - Bir değişken, programın farklı yerlerinde farklı adreslere sahip olabilir.
  - Aynı bellek konumuna erişmek için iki değişken adı kullanılabiliyorsa, bunlar takma ad (aliases) olarak adlandırılır.
  - Aliaslar, pointerlar, referans değişkenleri, C ve C++ unions aracılığıyla oluşturulur.
  - Aliaslar okunabilirliği olumsuz etkiler (program okuyucuları hepsini hatırlamak zorundadır).

Copyright © 2015 Pearson. All rights reserved.

5.3.1 Değişken Öznitelikleri (Variables Attributes)

- *Tür (Type)* - değişkenlerin değer aralığını ve bu tür değerler için tanımlanan işlem kümesini belirler; kayan nokta durumunda, tür aynı zamanda hassasiyeti de belirler.
- *Değer (Value)* - değişkenin ilişkili olduğu konumun içeriği
  - Bir değişkenin l-değeri, onun adresidir.
  - Bir değişkenin r-değeri, onun değeridir.
- *Soyut bellek hücresi (Abstract memory cell)* - bir değişkenle ilişkili fiziksel hücre veya hücreler topluluğu

Copyright © 2015 Pearson. All rights reserved.

5.4 Bağlayıcılık Kavramı (The Concept of Binding)

Bağlama (binding), bir entity ile bir attribute arasındaki ilişki, örneğin bir değişken ile onun türü veya değeri arasında ya da bir işlem ile bir sembol arasında olan ilişkidir.

- *Bağlanma süresi (Binding time)*, bağlanmanın gerçekleştiği zamandır.

Copyright © 2015 Pearson. All rights reserved.

5.4.1 Olası Binding Süreleri

- Dil tasarım zamanı -- işlem operatör sembollerini işlemlere bağlama
- Dil uygulama zamanı-- kayan nokta türünü bir temsil biçimine bağla
- Derleme zamanı -- C veya Java'da bir değişkeni bir türe bağlamak
- Yükleme zamanı -- C veya C++ `static` değişkenini bir bellek hücresine bağla)
- Çalışma zamanı -- statik olmayan yerel değişkeni bir bellek hücresine bağla

Copyright © 2015 Pearson. All rights reserved.

5.4.2 Static ve Dynamic Binding

- Binding, çalışma zamanından önce ilk kez ortaya çıkarsa ve programın yürütülmesi boyunca değişmezse *statiktir*.
- Binding, ilk olarak yürütme sırasında gerçekleşirse veya programın yürütülmesi sırasında değişebiliyorsa *dinamiktir*.

Copyright © 2015 Pearson. All rights reserved.

5.4.3 Type Binding

- Bir tür (type) nasıl belirtilir?
- Bağlayıcı işlem ne zaman gerçekleşir?
- Statik ise, tür açık veya örtük bir bildirimle belirtilebilir (If static, the type may be specified by either an explicit or an implicit declaration).

Copyright © 2015 Pearson. All rights reserved.

5.4.3.1 Açık/Örtük Bildirim (Explicit/Implicit Declaration)

- *Açık bir bildirim (explicit declaration)*, değişkenlerin türlerini bildirmek için kullanılan bir program ifadesidir.
- *Örtük bildirim (implicit declaration)*, bildirim deyimleri yerine varsayılan kurallar aracılığıyla değişken türlerini belirtmek için kullanılan varsayılan bir mekanizmadır.
- Basic, Perl, Ruby, JavaScript ve PHP örtük bildirimler sağlar.
  - Avantaj: yazılabilirlik (küçük bir kolaylık)
  - Dezavantaj: güvenilirlik (Perl ile daha az sorun)

Copyright © 2015 Pearson. All rights reserved.

5.4.3.1 Açık/Örtük Bildirim (Explicit/Implicit Declaration)

- Bazı diller, değişkenlerin türlerini (bağlam) belirlemek için tür çıkarımını kullanır.
  - C# - bir değişken **`var`** ve bir başlangıç değeri ile tanımlanabilir. Başlangıç değeri türü belirler.
  - Visual Basic 9.0+, ML, Haskell ve F# tür çıkarımını kullanır. Bir değişkenin göründüğü bağlam, onun türünü belirler.

Copyright © 2015 Pearson. All rights reserved.

5.4.3.2 Dinamik Tür Bağlama (Dynamic Type Binding)

- Dinamik Tür Bağlama (JavaScript, Python, Ruby, PHP ve C# (sınırlı))
- Atama deyimi ile belirtilir, örneğin JavaScript

```
list = [2, 4.33, 6, 8];
list = 17.3;
```

  - Avantaj: esneklik (genel program birimleri)
  - Dezavantajlar:
    - Yüksek maliyet (dinamik tip kontrolü ve yorumlama)
    - Derleyici tarafından tür hatası tespiti zordur.

Copyright © 2015 Pearson. All rights reserved.

5.4.4 Değişken Öznitelikleri (Variables Attributes)

- Storage Bindings & Lifetime
  - Allocation - kullanılabilir hücre havuzundan bir hücre alma
  - Deallocation - hücreyi havuza geri koymak
- Bir değişkenin ömrü (lifetime of a variable), belirli bir bellek hücresine bağlı olduğu süredir.

Copyright © 2015 Pearson. All rights reserved.

5.4.4 Ömürlerine Göre Değişken Kategorileri

- Static--yürütme başlamadan önce bellek hücrelerine bağlanır ve yürütme boyunca aynı bellek hücresine bağlı kalır, örneğin fonksiyonlardaki C ve C++ `static` değişkenleri
  - Avantajlar: verimlilik (doğrudan adresleme), geçmişe duyarlı alt program desteği
  - Dezavantajlar: esneklik eksikliği (yineleme (recursion) yok)

Copyright © 2015 Pearson. All rights reserved.

5.4.4 Ömürlerine Göre Değişken Kategorileri

- Stack-dynamic--Storage bindings, değişkenlerin deklarasyon ifadeleri ayrıntılı olarak ele alındığında oluşturulur..

  (Bir deklarasyon, onunla ilişkili yürütülebilir kod çalıştırıldığında ayrıntılı olarak açıklanır.)
- Skaler ise, adres hariç tüm öznitelikler statik olarak bağlanır.
  - C alt programlarındaki yerel değişkenler (`static` olarak tanımlanmamış) ve Java yöntemleri
- Avantaj: yinelemeye izin verir; depolama alanını korur
- Dezavantajlar:
  - Allocation ve deallocation giderleri
  - Alt programlar geçmişe duyarlı olamaz
  - Verimsiz referanslar (dolaylı adresleme)

Copyright © 2015 Pearson. All rights reserved.

5.4.4 Ömürlerine Göre Değişken Kategorileri

- *Explicit heap-dynamic* -- Programcı tarafından belirtilen ve yürütme sırasında etkili olan açık direktiflerle allocate edilir ve deallocate edilir.
- Yalnızca pointers veya referanslar aracılığıyla referanslanır, örneğin C++'daki dinamik nesneler (`new` ve `delete` aracılığıyla), Java'daki tüm nesneler
- Avantaj: dinamik depolama yönetimi sağlar
- Dezavantaj: verimsiz ve güvenilmez

Copyright © 2015 Pearson. All rights reserved.

5.4.4 Ömürlerine Göre Değişken Kategorileri

- *Implicit heap-dynamic*--Atama ifadeleriyle oluşan allocation ve deallocation
  - APL'deki tüm değişkenler; Perl, JavaScript ve PHP'deki tüm stringler ve arrayler
- Avantaj: esneklik (genel kod)
- Dezavantaj:
  - Verimsiz, çünkü tüm özellikler dinamiktir
  - Hata algılama kaybı

Copyright © 2015 Pearson. All rights reserved.

5.5 Variable Attributes: Scope

- Bir değişkenin *kapsamı* (*scope* of a variable), onun görünür olduğu ifadelerin aralığıdır.
- Bir program biriminin *local* değişkenleri, o birimde tanımlanan değişkenlerdir.
- Bir program biriminin yerel olmayan (*nonlocal*) değişkenleri, birimde görülebilen ancak orada tanımlanmayan değişkenlerdir.
- *Global* değişkenler, yerel olmayan değişkenlerin özel bir kategorisidir.
- Bir dilin kapsam kuralları, isimlere yapılan referansların değişkenlerle nasıl ilişkilendirileceğini belirler.

Copyright © 2015 Pearson. All rights reserved.

5.5.1 Static Scope

- Program metnine göre
- Bir isim referansını bir değişkene bağlamak için, siz (veya derleyici) deklarasyonu bulmalısınız.
- *Search process*: deklarasyonları, önce yerel olarak, ardından giderek daha geniş scope'larda, verilen ad için bir tane bulunana kadar arar.
- Statik scope'ları (belirli bir kapsama) çevreleyenler, *static ancestors* olarak adlandırılır; en yakın *static ancestor'a static parent* denir.
- Bazı diller, iç içe geçmiş statik scope oluşturan iç içe geçmiş alt program tanımlarına izin verir (örneğin, Ada, JavaScript, Common Lisp, Scheme, Fortran 2003+, F# ve Python).

Copyright © 2015 Pearson. All rights reserved.

5.5 Scope (Devam Ediyor)

- Değişkenler, aynı ada sahip bir “daha yakın” değişken kullanılarak bir birimden gizlenebilir.

Copyright © 2015 Pearson. All rights reserved.

5.5.2 Blocks

- Program birimleri içinde statik scope'lar oluşturma yöntemi - ALGOL 60'tan
- C'de bir örnek:

```
void sub() {
  int count;
  while (...) {
int count;
     count++;
     ...
  }
  …
}
```

- Not: C ve C++'da yasal, ancak Java ve C#'da değil - hataya çok açık

Copyright © 2015 Pearson. All rights reserved.

5.5.2.1 `LET` Yapısı (Construct)

- Çoğu fonksiyonel dil, bir tür `let` yapısı içerir.
- Bir let yapısı iki bölümden oluşur
  - İlk kısım isimleri değerlere bağlar
  - İkinci kısım, birinci kısımda tanımlanan isimleri kullanır.
- Scheme'de:

```
(LET (
   (name1 expression1)
   …
   (namen expressionn)
)
```

Copyright © 2015 Pearson. All rights reserved.

5.5.2.1 LET Yapısı (Construct)

- ML'de:

```
let
  val name1 = expression1
  …
  val namen = expressionn
in
 expression
end;
```

- F#'de:
  - İlk Kısım: `let` left_side = expression
  - (left_side bir isim veya bir tuple kalıbıdır
  - Aşağıdakiler ikinci kısımdır.

Copyright © 2015 Pearson. All rights reserved.

5.5.3 Deklarasyon Sırası (Declaration Order)

- C99, C++, Java ve C#, değişken deklarasyonun bir ifadenin görünebileceği her yerde görünmesine izin verir.
  - C99, C++ ve Java'da, tüm yerel değişkenlerin scope'u, deklarasyondan bloğun sonuna kadardır.
  - C#'da, bir blokta deklare edilen herhangi bir değişkenin scope'u, blok içindeki deklare konumundan bağımsız olarak tüm bloktur.
    - Ancak, bir değişken kullanılmadan önce yine de deklare edilmelidir.

Copyright © 2015 Pearson. All rights reserved.

5.5.3 Deklarasyon Sırası (Declaration Order)

- C++, Java ve C#'da değişkenler `for` ifadeleriyle tanımlanabilir.
  - Bu tür değişkenlerin scope'u `for` yapısı ile sınırlıdır.

Copyright © 2015 Pearson. All rights reserved.

5.5.4 Global Scope

- C, C++, PHP ve Python, bir dosyada bir dizi işlev tanımından oluşan bir program yapısını destekler.
  - Bu diller, değişken deklarasyonlarının fonksiyon tanımlarının dışında görünmesine izin verir.

- C ve C++ hem deklarasyonlara (sadece öznitelikler (attributes)) hem de tanımlara (öznitelikler ve depolama) sahiptir.
  - Bir fonksiyon tanımının dışında yer alan bir deklarasyon, fonksiyonun başka bir dosyada tanımlandığını belirtir.

Copyright © 2015 Pearson. All rights reserved.

5.5.4 Global Scope (Devam Ediyor)

- PHP
  - Programlar, HTML markup belgelerine, herhangi bir sayıda fragmentte, bazı ifadelerde ve bazı fonksiyon tanımlarında gömülüdür.
  - Bir fonksiyonda (“implicity”) bildirilen bir değişkenin scope’u, fonksiyonun yerel scope’udur.
  - Fonksiyonların dışında örtük (implicity) olarak bildirilen bir değişkenin kapsamı, bildirildiğinden programın sonuna kadardır, ancak araya giren fonksiyonları atlar.
    - Global değişkenlere bir fonksiyon içinde `$GLOBALS` dizisi aracılığıyla veya `global` olarak tanımlayarak erişilebilir.

Copyright © 2015 Pearson. All rights reserved.

5.5.4 Global Scope (Devam Ediyor)

- Python
  - Global değişkenler fonksiyonlarda referans alınabilir, ancak fonksiyon içinde `global` olarak tanımlanmışsa fonksiyon içinde atanabilir.

Copyright © 2015 Pearson. All rights reserved.

5.5.5 Static Scoping'in Değerlendirilmesi

- Çoğu durumda iyi sonuç verir
- Problemler:
  - Çoğu durumda, çok fazla erişim mümkündür.
  - Bir program geliştikçe, başlangıçtaki yapı bozulur ve lokal değişkenler genellikle global hale gelir; alt programlar da iç içe geçmiş olmaktan ziyade global hale gelme eğilimindedir.

Copyright © 2015 Pearson. All rights reserved.

5.5.6 Dynamic Scope

- Program birimlerinin metinsel düzenine değil, çağırma dizilerine göre (zamansal ve mekansal)
- Değişkenlere yapılan referanslar, bu noktaya kadar yürütmeyi zorlayan alt program çağrıları zincirini geriye doğru arayarak deklarasyonlara bağlanır.

Copyright © 2015 Pearson. All rights reserved.

5.5.6.1 Scope Örneği

```
function big() {
    function sub1()
       var x = 7;
    function sub2() {
      var y = x;
     }
   var x = 3;
 }
```

```
big calls sub1
sub1 calls sub2
sub2 uses x
```

- Static scoping
  - `sub2`'deki `x` referansı, `big`'in `x`'ine yöneliktir
- Dynamic scoping
  - `sub2`'deki `x` referansı, `sub1`'deki `x`'e yöneliktir

Copyright © 2015 Pearson. All rights reserved.

5.5.6.1 Scope Örneği

- Dynamic Scoping Değerlendirmesi:
  - Avantaj: kolaylık
  - *Dezavantaj:*
    1. Bir alt program çalışırken, değişkenleri çağırdığı tüm alt programlar tarafından görülebilir.
    2. Statik tip kontrolü imkansız
    3. Okunabilirlik zayıf - bir değişkenin türünü statik olarak belirlemek mümkün değildir

Copyright © 2015 Pearson. All rights reserved.

5.6 Scope ve Lifetime

- Scope ve lifetime bazen birbiriyle yakından ilişkili olmakla birlikte, farklı kavramlardır.
- C veya C++ fonksiyonundaki **`static`** bir değişkeni ele alalım.

Copyright © 2015 Pearson. All rights reserved.

5.7 Referencing Environments

- Bir ifadenin *referans ortamı (referencing environment)*, ifadede görünen tüm isimlerin (names) toplamıdır.
- Statik kapsamlı (static-scoped) bir dilde, lokal değişkenler ve tüm çevreleyen scope'taki görünür değişkenlerin tümüdür.
- Bir alt program, yürütülmesi başlamış ancak henüz sonlandırılmamışsa etkindir.
- Dinamik kapsamlı (dynamic-scoped) bir dilde, referans ortamı yerel değişkenler ve tüm aktif alt programlardaki görünür değişkenlerin tümüdür.

Copyright © 2015 Pearson. All rights reserved.

5.8 Named Constants

- *Adlandırılmış sabit (named constant)*, yalnızca depolama alanına bağlandığında bir değere bağlanan değişkendir.
- Avantajlar: okunabilirlik ve değiştirilebilirlik
- Programları parametrelendirmek için kullanılır
- Değerlerin named constant'lara bağlanması statik (manifest constants olarak adlandırılır) veya dinamik olabilir.
- Diller:
  - C++ ve Java: her türlü ifade, dinamik olarak bağlı
  - C#'da iki tür vardır: readonly ve const
    - const adlı sabitlerin değerleri derleme sırasında bağlanır.
    - `readonly` named constant değerleri dinamik olarak bağlanır.

Copyright © 2015 Pearson. All rights reserved.

Özet

- Büyük/küçük harf duyarlılığı ve names'lerin özel kelimelerle ilişkisi, names'lerin tasarımında önemli hususlardır.
- Değişkenler altı öğeden oluşur: name, address, value, type, lifetime, scope
- Binding, attributes'lerin program entities'leri ilişkilendirilmesidir.
- Skaler değişkenler şu şekilde sınıflandırılır: static, stack dynamic, explicit heap-dynamic, implicit heap-dynamic
- Strong typing, tüm tip hatalarını tespit etmek anlamına gelir.

Copyright © 2015 Pearson. All rights reserved.